

IBM Onboarding Platform

Architecture & Technical Design Document

1. Project Overview

The IBM Onboarding Platform is a full-stack web application designed to streamline and personalise the employee onboarding experience at IBM. It provides a unified digital portal for new joiners to track their onboarding journey, submit documents, receive AI-powered guidance, and communicate with their HR, manager, and buddy — all in one place. The platform is built as a two-role system: **Employee** (new joiners) and **Admin** (HR / onboarding managers).

2. Technology Stack

Layer	Technology	Purpose
Frontend	React 19 + Vite 8	Single-page application, component-based UI
Styling	Tailwind CSS 4	Utility-first responsive design system
Routing	React Router v7	Client-side navigation and role-based route guards
Animations	Framer Motion	Page transitions and micro-interactions
Auth (Client)	Firebase Auth SDK	Email/password authentication, ID token management
Auth (Server)	Firebase Admin SDK v14	Server-side ID token verification
Database	Cloud Firestore	Primary NoSQL document store for all application data
File Storage	Cloudinary	Document uploads (PDF, JPG, PNG) via upload_stream
Backend	Node.js + Express	REST API server, middleware pipeline
Real-time	Socket.IO	Live notifications pushed from server to employee client
AI Assistant	IBM Granite (watsonx.ai)	LLM powering the context-aware onboarding chatbot
AI Recommendations	IBM watsonx.ai API	Task and learning recommendations based on employee profile
Dev Server	Nodemon	Auto-restart backend on file changes

3. System Architecture

The platform follows a **client-server architecture** with a clear separation between the React frontend (port 5173) and the Express backend (port 5000). All persistent state is stored in Cloud Firestore. File assets are offloaded to Cloudinary. Real-time events are delivered via Socket.IO WebSocket connections.

Component Diagram

Browser (React SPA)	↔	Express API Server	↔	Cloud Firestore
Firebase Auth SDK	↔	Firebase Admin SDK	↔	Firebase Auth
Socket.IO Client	↔	Socket.IO Server		
Axios HTTP Client	→	REST Endpoints	→	Cloudinary CDN

4. Authentication Flow

- Step 1 — Signup: Client POSTs to /api/auth/signup → Express calls Firebase Admin auth.createUser() → Creates Firestore users/{uid} and employees/{uid} documents → Returns 201.
- Step 2 — Login: Client calls Firebase Auth signInWithEmailAndPassword() → Gets Firebase ID Token → POSTs idToken to /api/auth/login → Backend calls auth.verifyIdToken() → Looks up Firestore users/{uid} for role → Returns { token, user: { id, name, role } }.
- Step 3 — Session: Axios request interceptor calls firebaseUser.getIdToken() on every API request, attaches as Authorization: Bearer . Firebase tokens auto-refresh every hour.
- Step 4 — Middleware: authMiddleware.js on the backend verifies the Bearer token via Firebase Admin SDK, fetches the Firestore user doc to get role, attaches req.user = { id, role, email, name }.
- Step 5 — Role Guard: roleMiddleware.js checks req.user.role against the route's allowed roles. Frontend mirrors this with RoleRoute.jsx — admins redirect to /admin, employees to /dashboard.

5. Firestore Data Model

Collection	ID Strategy	Key Fields
users	Firebase Auth UID	name, email, role, status, createdAt
employees	Same UID as user (1:1)	user_id, department, designation, manager, buddy, joining_date, onboarding_s
documents	Auto Firestore ID	employee_id, document_name, document_type, cloudinary_url, verification_sta
checklists	Auto Firestore ID	employee_id, title, description, priority (high/medium/low), completed, complete
tasks	Auto Firestore ID	employee_id, title, description, assigned_by, status, deadline
accessRequests	Auto Firestore ID	employee_id, application_name, reason, status, approved_by
chatLogs	Auto Firestore ID	employee_id, sender (employee/assistant), message, timestamp

6. AI Chatbot Architecture — IBM Granite via watsonx.ai

The onboarding chatbot is powered by **IBM Granite**, accessed via the **IBM watsonx.ai REST API**. It follows a **RAG-lite (Retrieval-Augmented Generation)** pattern where the employee's live Firestore data is injected as context into the prompt before the LLM generates a response. This eliminates the need for a vector database since the employee profile is small and fully structured.

Chat Request Pipeline

Step	Action	Detail
1	Employee sends message	POST /api/chat { message } with Firebase Bearer token
2	Auth & role check	authMiddleware verifies token; roleMiddleware confirms 'employee' role
3	Fetch employee context	firestoreService fetches employees/{uid}, users/{uid}, documents, tasks, checklists
4	Build RAG prompt	Structured prompt: System instructions + Employee context object + Conversation history + User me
5	Call Granite LLM	POST to watsonx.ai /ml/v1/text/generation with model ibm/granite-13b-chat-v2

6	Parse & store	LLM response extracted → saved to chatLogs collection in Firestore
7	Return to client	{ success, response, timestamp } returned to React frontend

Data Sources for Context Injection

- **Firestore employees/{uid}**: Joining date, department, designation, manager, buddy, OS type, onboarding stage
- **Firestore users/{uid}**: Employee name, email, role
- **Firestore documents**: Document submission status and verification results
- **Firestore checklists**: Pending and completed checklist items
- **Firestore tasks**: Admin-assigned tasks and their statuses
- **Firestore accessRequests**: Tool access request statuses

Note: The current demo build uses a hardcoded employee context object and a keyword-intent engine to simulate LLM responses. In the production build, the context is fetched live from Firestore per authenticated user and passed to IBM Granite via the watsonx.ai API.

7. REST API Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/signup	Public	Create Firebase user + Firestore docs
POST	/api/auth/login	Public	Verify Firebase ID token, return role
GET	/api/auth/profile	Token	Return current user from Firestore
GET	/api/employee/profile	Employee	Get employee profile
POST	/api/employee/documents/upload	Employee	Upload doc to Cloudinary, store in Firestore
GET	/api/employee/documents	Employee	List employee documents
PUT	/api/employee/offer/accept	Employee	Accept offer, advance onboarding stage
GET	/api/employee/checklist	Employee	Get checklist + progress percentage
PATCH	/api/employee/checklist/:id	Employee	Toggle checklist item complete/incomplete
PUT	/api/employee/profile/os	Employee	Set OS preference
POST	/api/employee/access-request	Employee	Submit tool access request
GET	/api/employee/recommendations	Employee	Get AI-generated task recommendation
POST	/api/chat	Employee	Send message, get Granite AI response
GET	/api/chat/history	Employee	Retrieve last 100 chat messages
GET	/api/admin/stats	Admin	Dashboard counts
GET	/api/admin/employees	Admin	List all employees
PATCH	/api/admin/employees/:id/assign	Admin	Assign manager and buddy
POST	/api/admin/tasks	Admin	Assign task to employee
PATCH	/api/admin/documents/:id/verify	Admin	Verify or reject document
PATCH	/api/admin/access-requests/:id	Admin	Approve or reject access request

8. Real-time Notifications (Socket.IO)

- Employee client connects to the Socket.IO server on login and joins a private room: `employee_{uid}`.
- When an admin verifies a document, the server emits `employeeUpdated` to that employee's room.
- When an admin assigns a manager/buddy or a task, the employee receives a live notification banner.
- The employee dashboard listens for `employeeUpdated` events and auto-refreshes all data.
- Socket connection is torn down cleanly on logout via React's `SocketContext` cleanup effect.